

Ball Machine

Madina a inventat o mașină de aruncat mingi pentru a-i distra pe participanții la IOI. Structura internă a mașinii este următoarea: Mașina este formată din N **noduri**, indexate de la 0 la $N - 1$. Nodul $N - 1$ se numește **rădăcină**. Pentru fiecare nod u cu $0 \leq u < N - 1$, **părintele** nodului u este un nod $P[u]$ cu $P[u] > u$, iar nodul u este un **copil** al lui $P[u]$. Rădăcina nu are părinte. Un nod fără copii se numește **frunză**.

Nu cunoști N sau părintele $P[u]$, al vreunui nod u . În schimb, Madina îți spune M , numărul de frunze, și că indicii frunzelor sunt $0, 1, \dots, M - 1$. Sarcina ta este să determini structura internă a mașinii prin operarea acestora.

Fiecare nod al mașinii poate conține cel mult o **bilă**, iar fiecare bilă are o **valoare** care este un număr întreg non-negativ. Spunem că un nod are valoarea x dacă deține o bilă cu valoarea x . Un nod este **ocupat** dacă conține o bilă; altfel, este **liber**. Inițial, fiecare nod este liber.

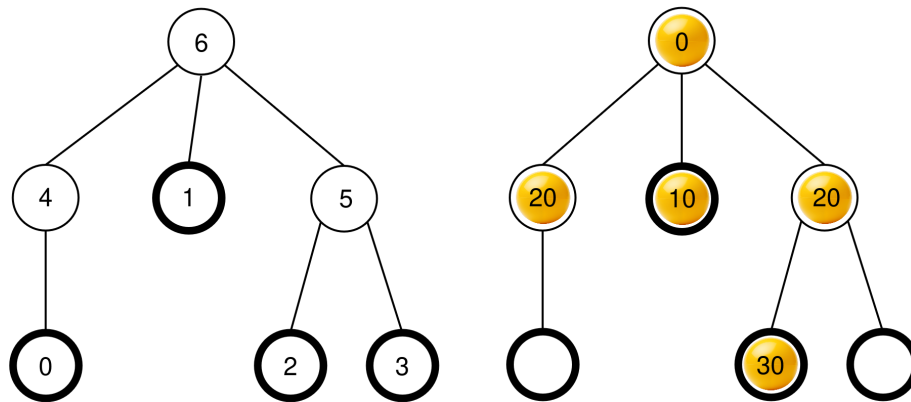
Puteți efectua două tipuri de operațiuni:

- **insert** (U, X) : încercă să insereze o nouă bilă cu valoarea X la frunza U . Dacă frunza U este ocupată, operația returnează `false` fără a insera bila. Dacă frunza U este liberă, bila este plasată în nodul U . Apoi, bila se mișcă în mod repetat către părintele nodului său curent, atâta timp cât acel părinte este liber. Mișcarea se oprește când bila ajunge la rădăcină sau la un nod al cărui părinte este ocupat. Operația returnează `true`.
- **collect()**: dacă rădăcina este liberă (ceea ce înseamnă că mașina este goală), `collect` returnează un tablou gol. În caz contrar, procedura recursivă `traverse` descrisă mai jos adună valorile tuturor bilelor care se află în prezent în mașină într-un tablou S . Inițial, tabloul S este gol și se apelează `traverse(N - 1)`.

```
traverse(u):  
    append the value of the ball at node u to the end of S  
    let c[u] be the list of the occupied children of u  
    sort c[u] in non-decreasing order according to the values  
      of the balls at those nodes (if multiple nodes have the  
      same value, they can appear in any order)  
    for each v in c[u]:  
        traverse(v)
```

După finalizarea acestei proceduri, **toate bilele sunt scoase** din mașină, iar `collect` returnează S .

De exemplu, să luăm în considerare o mașină cu $N = 7$ noduri și un tablou părinte $P = [4, 6, 5, 5, 6, 6]$. Imaginea din stânga afișează mașina goală cu indicii nodurilor, în timp ce imaginea din dreapta prezintă o posibilă configurație a bilelor după o oarecare secvență de operații de `insert` (această secvență este dată în secțiunea Exemplu).



Când se apelează `collect()`, rădăcina (nodul 6) este ocupată, deci S este inițializat ca $S = []$ și se apelează `traverse(6)`.

`traverse(6)` **: nodul 6 are valoarea 0; adăugarea sa la S dă $S = [0]$. Copiii nodului 6 sunt 4, 1 și 5, toți fiind ocupați. Valorile lor sunt 20, 10 și respectiv 20. Sortarea în ordine nedescrescătoare dă $c[6] = [1, 5, 4]$ (rețineți că $c[6] = [1, 4, 5]$ ar fi, de asemenea, valid, deoarece nodurile 4 și 5 au aceeași valoare). Procedura apelează apoi `traverse` pe fiecare nod din $c[6]$ în această ordine.

- **`traverse(1)`** : nodul 1 are valoarea 10; adăugarea lui la S dă $S = [0, 10]$. Nodul 1 nu are copii ocupați, deci acest apel se termină.
- **`traverse(5)`** : nodul 5 are valoarea 20; adăugarea lui la S dă $S = [0, 10, 20]$. Nodul 5 are un copil ocupat, nodul 2, deci $c[5] = [2]$ și procedura apelează `traverse(2)`.
 - **`traverse(2)`** : nodul 2 are valoarea 30; adăugarea lui la S dă $S = [0, 10, 20, 30]$. Nodul 2 nu are copii ocupați, deci acest apel se termină.
 - Execuția revine la `traverse(5)` care a procesat toți copiii din $c[5]$, deci apelul său se termină.
- **`traverse(4)`** : nodul 4 are valoarea 20; adăugarea lui la S dă $S = [0, 10, 20, 30, 20]$. Nodul 4 nu are copii ocupați, deci acest apel se termină.

Toate apelurile s-au finalizat și nu mai există noduri de procesat. În final, fiecare bilă este scoasă din mașină, iar `collect` returnează tabloul $S = [0, 10, 20, 30, 20]$.

Sarcina ta este să determini numărul de noduri N și să raportezi un tablou de părinți $R = [R[0], R[1], \dots, R[N - 2]]$ care descrie structura mașinii, dar cu o etichetare potențial diferită a nodurilor care nu sunt frunze și nu sunt rădăcina. Formal, un tablou raportat R este considerat corect dacă este posibil să se atribuie o etichetă distinctă $L[u]$ cu $0 \leq L[u] < N$ fiecărui nod u al mașinii astfel încât:

- $L[u] = u$ pentru orice $0 \leq u < M$ și $u = N - 1$, și
- $L[P[u]] = R[L[u]]$ pentru toate $0 \leq u < N - 1$.

Nu este obligatoriu ca $R[u] > u$. Mașina **trebuie să fie goală** când îți raportezi soluția.

Fie K numărul de operații `collect` efectuate, iar B valoarea maximă a oricărei bile în toate apelurile `insert`. Fie $C = K + B$. Atunci C **nu trebuie să depășească** 1000. Scorul obținut la unele subsarcini depinde de valoarea lui C .

Detalii de implementare

Ar trebui să implementați următoarea procedură.

```
std::vector<int> find_structure(int M)
```

- M : numărul de frunze din mașină.
- Procedura este apelată exact o singură dată pentru fiecare caz de testare.
- Procedura trebuie să returneze un tablou $R = [R[0], R[1], \dots, R[N - 2]]$ de lungime $N - 1$, un tablou corect de părinți.

Pentru a interacționa cu mașina, procedura dumneavoastră poate apela următoarele două proceduri.

Prima procedură este:

```
bool insert(int U, int X)
```

- U : indicele frunzei unde trebuie introdusă bila. Trebuie îndeplinită condiția $0 \leq U < M$.
- X : valoarea întreagă a mingii. Trebuie îndeplinită condiția $0 \leq X \leq 1000$.
- Această procedură returnează `true` dacă bila a fost introdusă cu succes sau `false` dacă frunza U era deja ocupată.
- Această procedură poate fi apelată de maximum 500 000.

A doua procedură este:

```
std::vector<int> collect()
```

- Colectează valorile tuturor bilelor introduse, golește mașina și returnează tabloul colectat S .

Dacă în orice moment al execuției programului valoarea lui C depășește 1000, soluția va primi verdictul `Output isn't correct: Too many resources used`.

Structura mașinii este fixată înainte de apelarea `find_structure`. Graderul este **determinist** în sensul că, dacă îl rulați de două ori și ambele rulări efectuează aceeași secvență de operații, apelurile către `collect` vor returna aceleași tablouri.

Restricții

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtask-uri

Subtask	Scor	Restricții suplimentare
1	5	Rădăcina are exact M copii.
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	Fără restricții suplimentare.

În subtask-urile 3 și 4, scorul depinde de valoarea lui C după cum urmează.

Subtask-ul 3 (25 de puncte)

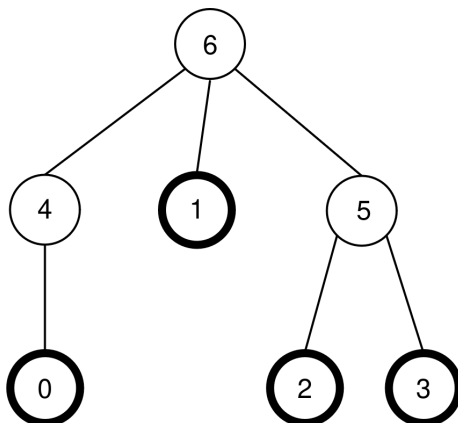
Limite	Scor
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask-ul 4 (60 de puncte)

Limite	Scor
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Exemplu

Luați în considerare aceeași mașină ca în descrierea sarcinii, deci $N = 7$, $M = 4$ și $P = [4, 6, 5, 5, 6, 6]$. Mașina este din nou prezentată în figura următoare:



Evaluatorul face următorul apel:

```
find_structure(4)
```

Procedura poate efectua următoarea secvență de apeluri:

1. **insert(0, 0)** : frunza 0 este liberă, deci o bilă cu valoarea 0 este plasată la frunza 0. Nodul $P[0] = 4$ este liber, deci bila se mișcă la nodul 4. Nodul $P[4] = 6$ este liber, deci bila se mișcă la nodul 6. Deoarece nodul 6 este rădăcina, mișcarea se oprește. Apelul returnează `true`.
2. **insert(3, 20)** : frunza 3 este liberă, deci o bilă cu valoarea 20 este plasată la frunza 3. Nodul $P[3] = 5$ este liber, deci bila se mișcă la nodul 5. Deoarece $P[5] = 6$ este ocupat, mișcarea se oprește. Apelul returnează `true`.
3. **insert(1, 10)** : frunza 1 este liberă, deci o bilă cu valoarea 10 este plasată la frunza 1. Deoarece $P[1] = 6$ este ocupată, bila rămâne la nodul 1. Apelul returnează `true`.
4. **insert(2, 30)** : frunza 2 este liberă, deci o bilă cu valoarea 30 este plasată la frunza 2. Deoarece $P[2] = 5$ este ocupată, bila rămâne la nodul 2. Apelul returnează `true`.

5. `insert(1, 25)` : frunza 1 este ocupată, deci bila nu este plasată la frunza 1. Apelul returnează `false`.
6. `insert(0, 20)` : frunza 0 este liberă, deci o bilă cu valoarea 20 este plasată la frunza 0. Nodul $P[0] = 4$ este liber, deci bila se mișcă la nodul 4. Deoarece $P[4] = 6$ este ocupat, mișcarea se oprește. Apelul returnează `true`.

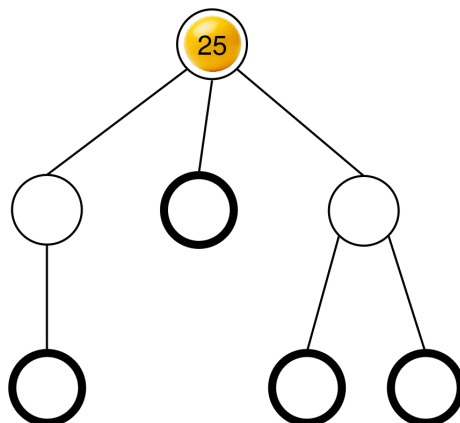
Configurația bilelor rezultată a fost deja afișată în descrierea sarcinii.

În continuare, procedura poate apela:

```
collect()
```

Executarea acestui apel a fost deja explicată în descrierea sarcinii, așadar acest apel returnează $S = [0, 10, 20, 30, 20]$. Ulterior, mașina este din nou goală.

Apoi, procedura poate apela `insert(2, 25)`. Frunza 2 este liberă, așadar o bilă cu valoarea 25 este plasată la frunza 0. Această bilă se mută apoi la nodul 5, și apoi la nodul 6, unde se oprește. Apelul returnează `true`. Configurația bilelor rezultată este prezentată în figura următoare.



În final, procedura poate apela `collect()` care returnează $S = [25]$ și golește mașina.

Există două tablouri părinte corecte pe care le poate returna `find_structure` : $R = P = [4, 6, 5, 5, 6, 6]$ (ceea ce corespunde etichetării $L = [0, 1, 2, 3, 4, 5, 6]$) și $R = [5, 6, 4, 4, 6, 6]$ (ceea ce corespunde etichetării $L = [0, 1, 2, 3, 5, 4, 6]$). În acest exemplu, $K = 2$ și $B = 30$, deci $C = 32$.

Grader local

Format de intrare:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Format de ieşire:

```
K B C
QR[0] R[1] R[2] ... R[Q-1]
```

Aici, Q este lungimea tabloului R returnat de `find_structure`.